

NAME: M S MUKILAN

REG NO:192412071

COURSE: ITA0609-MACHINE LEARNING

CO2 – AT3: Scenario Based Questions

1.Backpropagation in a Multilayer Perceptron (5 Marks)

A Multilayer Perceptron (MLP) used for binary classification typically consists of an input layer, one or more hidden layers, and an output layer with a single neuron (using a sigmoid activation) that outputs a probability between 0 and 1. The network learns by repeatedly performing a forward pass, calculating the error, and then propagating that error backward to update the weights — this is the backpropagation algorithm.

1.1 Overview of the Backpropagation Process

- **Forward Pass:** Input values are multiplied by weights, summed with a bias, and passed through an activation function at each layer to produce an output.
- **Error Calculation:** The predicted output is compared with the actual (target) output using a loss function such as Mean Squared Error or Binary Cross-Entropy.
- **Backward Propagation:** The error is propagated backward from the output layer to the hidden and input layers using the chain rule of calculus, computing the gradient of the loss with respect to each weight.
- **Weight Update:** Each weight is adjusted in the direction that reduces the error, scaled by the learning rate.

1.2 Illustrative Example

Consider a simple MLP with two inputs (x_1, x_2), one hidden layer with two neurons (h_1, h_2), and one output neuron (o), all using the sigmoid activation function.

Step 1 — Forward Pass

Each hidden neuron computes a weighted sum of the inputs plus a bias, then applies the sigmoid function:

$$net_h1 = (w1 \cdot x1 + w2 \cdot x2) + b1, \quad h1 = \text{sigmoid}(net_h1)$$

$$net_h2 = (w3 \cdot x1 + w4 \cdot x2) + b2, \quad h2 = \text{sigmoid}(net_h2)$$

The outputs of the hidden neurons are then combined at the output layer:

$$net_o = (w5 \cdot h1 + w6 \cdot h2) + b3, \quad \hat{y} = \text{sigmoid}(net_o)$$

Here $\hat{y}$ is the network's predicted probability that the example belongs to the positive class (e.g., "Yes").

Step 2 — Error Calculation

The error (loss) is computed by comparing the predicted output $\hat{y}$ with the actual target y (0 or 1). A common choice is the squared error:

$$E = \frac{1}{2} (y - \hat{y})^2$$

For example, if the actual output $y = 1$ and the network predicts $\hat{y} = 0.7$, the error indicates the network under-predicted the positive class, and this error must be propagated backward to correct the weights.

Step 3 — Backward Propagation of Error

Using the chain rule, the gradient of the error with respect to each weight is calculated, starting at the output layer and moving backward:

$$\delta_o = (\hat{y} - y) \cdot \hat{y} \cdot (1 - \hat{y}) \quad [\text{output layer gradient, using the sigmoid derivative}]$$

$$\delta_h = \delta_o \cdot w \cdot h \cdot (1 - h) \quad [\text{hidden layer gradient, back-propagated through weight } w]$$

These delta (δ) terms represent how much each neuron contributed to the overall error.

Step 4 — Weight Update

Each weight is then updated in proportion to its gradient and the learning rate (η):

$$w_{new} = w_{old} - \eta \cdot (\partial E / \partial w)$$

This process — forward pass, error calculation, backward pass, and weight update — is repeated over many epochs until the error converges to an acceptably low value.

1.3 Influence of Learning Rate and Activation Functions

- **Learning Rate:** A learning rate that is too high can cause the weight updates to overshoot the minimum of the loss function, leading to oscillation or divergence. A learning rate that is too low results in very slow convergence and the model may get stuck in local minima or take excessively long to train.
- **Activation Functions:** Sigmoid and tanh functions can suffer from the vanishing gradient problem in deep networks, since their derivatives become very small for large positive or negative inputs, slowing down learning in earlier layers. ReLU (Rectified Linear Unit) mitigates this problem and generally speeds up convergence, though it can suffer from "dying neurons" if too many activations become zero.

- Together, an appropriately chosen learning rate (often tuned using techniques like learning rate schedules or adaptive optimizers such as Adam) and a suitable activation function (e.g., ReLU in hidden layers, sigmoid in the output layer for binary classification) allow the network to converge faster and achieve higher classification accuracy.

2. Genetic Algorithm for Maximizing a Fitness Function (5 Marks)

Genetic Algorithms (GAs) are search and optimization techniques inspired by natural selection. A population of candidate solutions evolves over generations through selection, crossover, and mutation, gradually converging toward an optimal or near-optimal solution.

2.1 Problem Setup and Dataset

Consider a scenario where each candidate solution is defined by two parameters — Speed and Accuracy, both encoded as values from 0 to 9 (which can be represented in binary as 4-bit strings, e.g., Speed = 6 $\rightarrow$ 0110). The fitness function is defined as:

Fitness = Speed + Accuracy (higher is better; a solution is "suitable" if $Fitness \geq 14$)

Initial Population (Generation 0)

Chromosome ID	Speed	Accuracy	Chromosome (Speed Accuracy)	Fitness	Suitable
C1	5	7	0101 0111	12	No
C2	8	6	1000 0110	14	Yes
C3	3	4	0011 0100	7	No
C4	7	9	0111 1001	16	Yes

2.2 Selection

Selection identifies the fittest individuals to act as parents for the next generation, typically using roulette-wheel or tournament selection, where individuals with higher fitness have a proportionally higher chance of being selected. Based on the fitness values above, C4 (fitness = 16) and C2 (fitness = 14) are the strongest candidates and are selected as parents, since they already meet the suitability threshold, while low-fitness individuals such as C3 are eliminated.

2.3 Crossover

Crossover combines genetic material from two parent chromosomes to produce offspring, introducing new combinations of traits. Using single-point crossover on the parents C2 (1000|0110) and C4 (0111|1001) at the midpoint of the chromosome:

Parent	Chromosome
C2 (Parent 1)	1000 0110
C4 (Parent 2)	0111 1001
Offspring O1	1000 1001
Offspring O2	0111 0110

Decoding the offspring: O1 → Speed = 8, Accuracy = 9 → Fitness = 17. O2 → Speed = 7, Accuracy = 6 → Fitness = 13. O1 already exceeds both parents in fitness, showing how crossover can generate stronger offspring by recombining favorable traits.

2.4 Mutation

Mutation introduces small random changes (typically flipping a bit) to maintain genetic diversity and allow the algorithm to explore regions of the search space that selection and crossover alone might miss, helping avoid premature convergence to a local optimum. For example, flipping the last bit of O2's Accuracy gene (0110 → 0111) changes Accuracy from 6 to 7:

O2 (mutated): Speed = 7, Accuracy = 7 → Fitness = 14 (now "Suitable")

Generation 1 — Resulting Population

Chromosome ID	Speed	Accuracy	Fitness	Suitable
O1	8	9	17	Yes
O2 (mutated)	7	7	14	Yes
C2 (retained)	8	6	14	Yes
C4 (retained)	7	9	16	Yes

2.5 Evolution Across Generations and Optimal Solution

Across successive generations, the average and maximum fitness of the population steadily increase as weaker individuals (like C3, fitness = 7) are replaced by fitter offspring (like O1, fitness = 17).

Repeating selection, crossover, and mutation over further generations continues to push the population toward chromosomes with Speed and Accuracy close to their maximum values.

Final Optimal Solution: Chromosome O1, with Speed = 8 and Accuracy = 9 (Fitness = 17), emerges as the best solution found in this evolution, closely approaching the theoretical maximum fitness of 18 (Speed = 9, Accuracy = 9). Further generations would be expected to converge even closer to this global optimum.

3. Neural Network for Disease Classification (5 Marks)

In this scenario, a neural network is trained to predict whether a patient has a disease (Yes/No) based on four input attributes: Age, Blood Pressure, Glucose Level, and Lifestyle Risk. Categorical attributes (Blood Pressure, Glucose Level, Lifestyle Risk) are first encoded numerically (e.g., Low = 0, Normal/Medium = 1, High = 2) and Age is normalized, before being fed into the network.

3.1 Given Dataset

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

3.2 Training with Backpropagation

The four attributes form the input layer of the network. During training, each example is passed through the network (forward pass); the predicted output is compared against the actual label using a loss function (e.g., binary cross-entropy); and the resulting error is propagated backward through the hidden layer(s) to the input layer, updating all weights via gradient descent, exactly as described in Question 1. This process is repeated over many epochs across all five (and typically many more) training examples until the network's predictions closely match the actual outputs, at which point the weights are considered trained.

3.3 Confusion Matrix

Comparing the Actual Output and Predicted Output columns for each example gives the following classification outcomes:

Example	Actual	Predicted	Outcome
1	Yes	Yes	True Positive (TP)
2	Yes	No	False Negative (FN)
3	No	No	True Negative (TN)
4	Yes	Yes	True Positive (TP)
5	No	Yes	False Positive (FP)

This gives: TP = 2, TN = 1, FP = 1, FN = 1.

3.4 Performance Evaluation Metrics

Metric	Formula	Calculation	Value
Accuracy	$(TP + TN) / \text{Total}$	$(2 + 1) / 5$	0.60 (60%)
Precision	$TP / (TP + FP)$	$2 / (2 + 1)$	0.67 (67%)
Recall (Sensitivity)	$TP / (TP + FN)$	$2 / (2 + 1)$	0.67 (67%)
F1-Score	$2 \times (P \times R) / (P + R)$	$2 \times (0.67 \times 0.67) / (1.34)$	0.67 (67%)

With only 5 examples, the model achieves 60% accuracy and 67% precision/recall — Example 2 (a missed "Yes" case, i.e., a false negative) and Example 5 (a false alarm, i.e., a false positive) indicate that the current network has not yet fully learned the relationship between the input attributes and the disease outcome, and would likely benefit from more training data and further tuning.

3.5 Role of Preprocessing and Feature Selection

- **Data Normalization/Encoding:** Scaling numeric attributes like Age and consistently encoding categorical attributes (High/Normal, High/Medium/Low) prevents attributes with larger numeric ranges from dominating the learning process and helps the network converge faster.
- **Handling Missing/Noisy Data:** Cleaning the dataset of missing or inconsistent values (e.g., ambiguous Lifestyle Risk labels) prevents the network from learning incorrect patterns.
- **Feature Selection:** Identifying which attributes (Age, Blood Pressure, Glucose Level, Lifestyle Risk) most strongly influence the disease outcome — and removing irrelevant or redundant features — reduces model complexity, lowers the risk of overfitting, and improves generalization to new patients.
- **Balanced Data:** Ensuring the training set has a reasonable balance of "Yes" and "No" cases (unlike this small 3:2 sample) helps the network avoid bias toward the majority class and improves both recall and overall reliability.

Together, careful preprocessing and thoughtful feature selection improve both the accuracy of the trained neural network and the reliability of its predictions on unseen patient data.